Annexe A : Sujets bonus

Introduction

À ce stade, vous en savez assez pour commencer à utiliser git de façon efficace au travail en apprenant le reste par vous-même. Mais si vous souhaitez en savoir plus, ce TP complémentaire peut vous présenter plusieurs sujets plus avancés.

Contrairement aux TPs précédents, il n'y a pas d'ordre, ni de marche à suivre précise. Prenez les différentes « parties » dans l'ordre que vous souhaitez, et essayez ces nouvelles fonctionnalités sur des dépôts git de votre choix, par exemple ceux que vous avez créés au cours de la formation !

La double vie de `git mv`

Au cours du premier TP, nous avons présenté `git mv` comme une commande qui déplace un fichier sous gestion de version. En réalité, ce n'est pas exactement ce qui se passe.

Lorsqu'on exécute la commande `git mv`, git effectue sous le capot les opérations suivantes :

- Déplacer le fichier au sein du répertoire de travail
- Noter que le fichier a été supprimé de son ancien emplacement
- Noter qu'un fichier a été ajouté à son nouvel emplacement

Un mouvement de fichier est donc traité exactement comme une suppression suivie d'une re-création, et c'est ensuite une optimisation de performance interne à git qui s'assure que cette opération soit implémentée de façon efficace.

C'est équivalent, me direz-vous. En fait, c'est même mieux, parce que cela permet à git de traiter plus efficacement des opérations qui sont *presque* des déplacements (ex : déplacement avec ajout mineur de contenu). Cependant, cela conduira aussi parfois les outils construits autour de git à échouer à identifier un déplacement de fichier lorsque cela leur serait utile.

Identifier l'origine d'un problème

L'outil `git bisect` permet de localiser rapidement l'origine d'un problème dans l'historique du dépôt en utilisant une recherche par dichotomie. Il est d'autant plus efficace que les recommandations données précédemment concernant les commits (peu de changements par commit, et des commits auto-suffisants) ont bien été suivies.

Pour utiliser cet outil, il faut procéder de la façon suivante. D'abord, on active la recherche par dichotomie :

```
$ git bisect start
```

Ensuite, on désigne un commit incorrect avec `git bisect bad [<commit>]`. Si on ne spécifie pas quel commit est mauvais, c'est le commit courant qui sera sélectionné par défaut :

```
$ git bisect bad
```

Puis on désigne un commit dans le passé qui était correct, avec la commande `git bisect good [<commit>]`, qui fonctionne comme `git bisect bad` :

```
? git bisect good <commit correct>
```

À partir de là, git va commencer à chercher en quel point de l'historique les choses ont mal tourné, via une recherche par dichotomie. À chaque commit où `git bisect` s'arrête, il vous sera demandé de dire si ce commit est correct ou non, avec `git bisect good` ou `git bisect bad`. Et au bout d'un faible nombre d'itérations, git vous indiquera quel commit semble être à l'origine du problème que vous étudiez.

Si pour une raison ou une autre (typiquement du code qui ne compile pas) vous ne pouvez pas dire si un commit est bon ou pas selon le critère que vous étudiez actuellement, vous pouvez utiliser `git bisect skip` pour exprimer cette incertitude.

Il est aussi possible d'automatiser complètement `git bisect` en utilisant `git bisect run <commande>`, qui prend en argument un programme capable de vérifier si le commit actif est correct ou non. Consultez la documentation de `git bisect` pour plus d'informations sur cette fonctionnalité.

Vous trouverez dans les supports de la formation deux scripts pour vous entraîner à utiliser `git bisect`. Consultez le README des supports pour plus d'informations sur leur utilisation.

Chercher une information dans le dépôt

Vous êtes sans doute familier avec la commande `grep`, qui permet de rechercher un texte au sein d'une hiérarchie de fichiers. git fournit une alternative à celle-ci, `git grep`, qui...

1. Tire parti des informations connues de git pour effectuer la recherche beaucoup plus rapidement, ce qui est précieux dans les gros dépôts.
2. Est capable de rechercher des informations non seulement dans le répertoire de travail, mais aussi dans l'historique des commits.
3. Se restreint aux fichiers sous gestion de version.

En contrepartie, cette commande nécessite un temps d'adaptation pour l'habitué de GNU grep, car sa logique est un peu différente. Par exemple, le dialecte d'expression régulière par défaut n'est pas le même.

La manière la plus simple d'utiliser `git grep` est de rechercher un texte exact dans le dépôt :

```
$ git grep 'Une recherche passionnante'
```

Une option très utile est `-E`, qui active le support des expressions régulières Perl pour permettre de faire des recherches plus flexibles. Le prix à payer étant qu'il y a davantage de caractères spéciaux que l'on doit parfois « échapper » avec \ dans la chaîne recherchée. Voici un exemple qui permet de trouver toutes les occurrences de « ceci », « cela » et « ça » :

```
$ git grep -E '(ce(ci|la)?|ça)'
```

Guide complet de `reset` et `checkout`

Parmi les commandes git d'usage courant, les plus difficiles à maîtriser sont sans doute `git reset` et `git checkout`. Cela tient au fait qu'elles font un grand nombre de choses différentes selon ce qu'on leur passe en paramètre. Passons donc au crible leurs différentes variantes.

`git checkout <commit>` est une commande qui sert à se promener dans l'historique. Elle bascule `HEAD` sur un commit donné, et met à jour l'index et le répertoire de travail pour qu'ils collent au contenu de ce commit. Il vaut donc mieux l'exécuter avec un répertoire de travail propre, sinon elle échouera avec un message d'erreur.

Cette commande change également de branche active. Si on lui passe une branche en paramètre, ladite branche devient la branche active, au sein de laquelle les nouveaux commits sont enregistrés. Si on passe à `checkout` quelque chose qui n'est pas une branche (une étiquette, un *hash* de commit…), alors il n'y aura plus de branche active. On parlera de « `HEAD` détachée ».

Comme nous l'avons vu au TP 4, cette commande peut aussi créer automatiquement une branche locale liée à une branche distante.

Toujours dans les raccourcis, `git checkout -b <nom>` est une abréviation de `git branch <nom> && git checkout <nom>` : on crée une branche et on bascule dessus.

`git checkout <fichiers>` modifie des fichiers du répertoire de travail pour qu'ils correspondent à leur version dans l'index. Vous pouvez le voir comme l'inverse de la commande `git add <fichiers>`, qui modifie l'index pour qu'il corresponde au contenu du répertoire de travail.

`git checkout <commit> -- <fichiers>` sert à récupérer et mettre dans le répertoire de travail et l'index des versions de fichier issues d'un certain commit.

La branche active et le pointeur `HEAD` ne sont pas affectés par ces deux dernières formes de `git checkout`. Comme `git add`, `git checkout` supporte aussi une option `--patch` qui permet de récupérer des changements depuis la source choisie à une granularité plus fine que le fichier.

Voilà pour `git checkout`. Maintenant, passons à `git reset`.

`git reset <commit>` est une commande qui sert à déplacer la branche active (et `HEAD`) sur un autre commit. Elle se distingue en cela de `checkout`, qui change `HEAD` mais ne déplace pas la branche active.

Avec cette commande, il est possible de rendre des commits inaccessibles. Assurez-vous donc d'avoir bien compris comment utiliser les *reflogs* avant d'utiliser cette variante de `git reset`, afin de pouvoir facilement ramener votre branche au point précédent en cas de manipulation incorrecte.

Cette commande possède trois variantes, correspondant à différents niveaux d'agressivité vis-à-vis de l'index et du répertoire de travail :

- `git reset --soft <commit>` déplace uniquement la branche active, en laissant l'index et le répertoire de travail tels quels. Comme on l'a vu précédemment, on peut notamment s'en servir pour annuler un appel prématuré à `git commit`.
- `git reset [--mixed] <commit>`, qui est le mode par défaut, met également à jour l'index pour qu'il corresponde au contenu du nouveau `HEAD`. Il n'y a donc plus de « changements prêts à être commités » dans `git status` après exécution de cette commande.
- `git reset --hard <commit>` propage également les versions de fichiers de `HEAD` au sein du répertoire de travail. **Cette opération peut donc perdre des données présentes au sein de ce dernier de façon irréversible**, et il faut l'utiliser avec précaution._ Mais c'est aussi l'un des moyens les plus simples de « ramener une branche en arrière » après des commits incorrects.
  - On peut mentionner les variantes un peu plus sûres `--merge` et `--keep` qui refusent d'écraser des changements non versionnés du répertoire de travail, et diffèrent dans leur façon de traiter l'index. `--keep` préserve les changements indexés, là où `--merge` les efface.

`git reset [<commit>] -- <fichiers>` récupère la version des fichiers choisis du commit choisi (`HEAD` par défaut) et l'extrait dans l'index. L'utilisation la plus courante de cette variante est d'annuler l'effet de `git add` en ramenant certains fichiers de l'index au point où ils étaient dans `HEAD`.

Il existe un parallèle important entre cette dernière commande et la variante de `git checkout` qui prend des noms de fichiers en paramètre. La seule différence entre les deux est qu'on opère sur l'index au lieu d'opérer sur le répertoire de travail. Ici aussi, l'option `--patch` permet de contrôler plus finement quels changements on effectue au sein de l'index.

De l'avis général, ces deux commandes font un peu trop de tâches différentes pour être facilement compréhensibles. Des commandes plus spécialisées ont donc été introduites récemment dans `git` pour faciliter des tâches courantes.

Ces commandes sont encore considérées comme expérimentales, donc il est possible qu'à l'heure où vous lirez ces lignes, la description qui suit ne soit plus à jour, mais en voici les grandes lignes :

- `git switch` est une forme simplifiée de `git checkout <commit>` qui permet uniquement de basculer entre des branches. Par défaut, cette commande refuse de basculer sur un commit ou un *tag* isolé, ce qui nous placerait dans un état de « `HEAD` détachée », mais on peut indiquer que c'est ce qu'on veut avec l'option `--detach` qui s'abbrévie en `-d`.
- `git restore` combine les fonctions des variantes de `git checkout` et `git reset` qui prennent des fichiers en paramètre. Cette commande permet…
  - Dans son mode par défaut, de remplacer des fichiers du répertoire de travail par la version qui existe au sein de l'index. Si vous avez effectué des changements incorrects au sein de votre répertoire de travail depuis votre dernier `git add`, cela vous permet de revenir au point où vous en étiez lors de votre `git add`.
  - Avec l'option `--staged`, de récupérer au sein de l'index des versions de fichiers issues de `HEAD`. C'est par exemple comme ça qu'on annulerait l'effet de `git add` ou `git rm`.
    - Si l'on souhaite aussi récupérer les versions de fichiers au sein du répertoire de travail, donc annuler toutes les modifications d'un fichier effectuées au sein du répertoire de travail depuis `HEAD`, on peut utiliser `--staged --worktree`.
  - En ajoutant l'option `--source <commit>`, on récupère plutôt au sein du répertoire de travail (ou de l'index si `--staged` est spécifié) des fichiers issus d'anciens commits.

Savoir d'où vient une partie d'un fichier

Parfois, il arrive que l'on souhaite savoir qui est l'auteur d'une partie du texte présent au sein du dépôt. Comme c'est rarement pour faire des éloges à cette personne, la commande pour le faire s'appelle `git blame`.

Quand on exécute `git blame <fichier>`, git affiche une version annotée du fichier en question, comprenant le *hash* du dernier commit où ce fichier a été modifié, la date de modification, et le nom de l'auteur de cette dernière.

On peut ensuite afficher la nature exacte des modifications qui ont été effectuées, par exemple avec `git show <commit>` ou en faisant un *checkout* sur le commit correspondant. Cela permettra de vérifier que la personne est bien l'auteur initial de la partie du fichier concerné. En effet, le moindre reformatage d'un document marquera du point de vue de git la personne qui a effectué l'opération comme l'auteur de la section reformatée…

Édition d'historique avec le *rebase* interactif

Pour terminer en beauté ce petit tour des fonctionnalités avancées de git, mentionnons un outil aussi dangereux que puissant, qui permet de modifier l'historique des commits d'une façon beaucoup plus profonde que tout ce que nous avons vu jusqu'à présent : `git rebase --interactive <commit>`.

Pour rappel, un *rebase* est une opération consistant à rejouer les commits d'une branche par-dessus un nouveau commit parent. Lorsque nous avons introduit cette fonction, nous l'avons présentée comme un moyen de « mettre à jour » une branche quand sa branche parente a reçu de nouveaux commits.

Mais lorsqu'on ajoute y l'option `--interactive`, ou son raccourci `-i`, git nous permet en plus d'intervenir sur le processus par lequel les commits de la branche sont rejoués, et d'agir dessus de différentes manières. On peut ainsi...

- Modifier la description de certains commits de la branche
- Modifier le contenu de ces commits (nouveaux fichiers, suppressions...)
- Fusionner plusieurs commits en un seul
- Supprimer purement et simplement des commits de l'historique

Il va de soi que c'est une technique relativement dangereuse, dont il faut user avec précaution. En plus des risques habituels du *rebase*, la possibilité de modifier le contenu de commits et de supprimer ces derniers peut aussi conduire à éliminer accidentellement des changements d'une branche.

Ces pertes de données ne sont qu'apparentes, et il est possible d'annuler un *rebase* interactif, soit en créant préalablement un tag de sauvegarde à l'ancien emplacement de la branche et en faisant un `git reset --hard` dessus en cas de problème, soit à la force des poings en retrouvant le point du *reflog* où se trouvait la branche avant l'exécution pour effectuer un `reset` dessus.

Mais il n'en reste pas moins que réparer les dégâts d'un *rebase* interactif incorrect demande une bonne maîtrise de git, et qu'il est en ce sens quelque peu regrettable que certains projets logiciels forcent l'usage de ces techniques relativement avancées chez leurs contributeurs.

Git LFS (*Large File Storage*)

Dans sa conception, git est surtout optimisé pour stocker des documents textuels. Son mécanisme de stockage est optimisé pour des fichiers texte bruts dont peu de lignes changent à chaque *commit*, et des outils comme `git diff` et `git merge` ne fonctionnent bien que dans ce contexte.

Mais dans la vraie vie, on n'utilise pas que du texte brut. Un dépôt git peut tout aussi bien se retrouver lié à des fichiers binaires tels que des documents texte enrichis, des images, des archives compressées, ou même des images de machines virtuelles.

Pour ce genre d'usage, git va assez mal se comporter : le calcul de différences ne va plus marcher, ce qui signifie que...

- Toute fusion de branches où les deux branches source ont touché au fichier binaire sera considérée comme un conflit, ce qui limite les possibilités de collaboration.
- À chaque modification d'un fichier binaire, on en stockera une nouvelle copie complète, ce qui peut prendre beaucoup de place sur le disque et ralentir les échanges réseau.

Pour limiter l'impact de ce second problème, les équipes de la forge GitHub ont mis au point une extension de git appelée Git *Large File Storage* (généralement abrévié en Git LFS), qui permet de s'assurer que git ne stocke plus l'ensemble des versions des fichiers binaires, mais seulement celles dont on a eu besoin à un moment donné. Lorsqu'une nouvelle version est requise, elle est téléchargée depuis le serveur.

Les conséquences sont les suivantes :

- Côté serveur, peu de changements, on stocke toujours toutes les versions du fichier et il faut donc prévoir de l'espace de stockage en conséquence.
- Côté client, on ne possède plus une copie complètement indépendante du dépôt git, le `checkout` d'anciens commits peut nécessiter une connexion internet.
- Il faut que le serveur (la forge) soit compatible avec LFS, et que chaque utilisateur du dépôt ait installé l'extension git LFS.

Insistons un peu sur ce dernier point, car il est important. Si tous les utilisateurs du dépôt ne sont pas équipés de l'extension Git LFS, alors il est très facile de se retrouver dans une situation où les gros fichiers sont remplacés par des simples identifiants textuels (utilisés par Git LFS en interne). Il est donc important que la décision d'utiliser Git LFS et les prérequis associés soient communiqués clairement à l'ensemble de l'équipe.

Vous trouverez plus d'information et des instructions d'installation à l'adresse https://git-lfs.github.com/.

Submodules et alternatives

Il est courant de se retrouver dans une situation où l'on a un dépôt git A qui dépend du contenu d'un autre dépôt git B. Dans ce chapitre, nous allons utiliser pour clarifier les choses l'exemple du développement logiciel d'une application qui dépend d'une bibliothèque externe pour fonctionner, mais le propos est généralisable à d'autres situations.

Dans ce type de situation, le principal problème que l'on doit gérer est qu'en général, toutes les versions du dépôt A ne sont pas compatibles avec toutes les versions du dépôt B. Dans notre exemple...

- Le dépôt B peut, à un certain point dans son historique, introduire une nouvelle fonctionnalité que A va utiliser par la suite, ce qui va rendre les nouvelles versions A incompatible avec les anciennes versions de B.
- Le dépôt B peut modifier une de ses interfaces de telle façon que A ne peut plus l'utiliser tel quel. À ce moment-là, A ne sera plus compatible avec les nouvelles versions de B. Par la suite, le dépôt A pourra basculer sur la nouvelle interface de B, ce qui rendra ses nouvelles versions compatibles avec les nouvelles versions de B mais incompatibles avec les anciennes versions.

Afin de bien gérer cette notion de compatibilité entre versions, on doit avoir un moyen de spécifier, à chaque version du dépôt A, une ou plusieurs versions du dépôt B avec lesquelles celui-ci est compatible. Il existe plusieurs manières classiques de le faire, chacune avec ses avantages et ses inconvénients :

1. On peut fusionner les deux dépôts en un seul, et développer la bibliothèque B au sein du dépôt A. Quand cette approche est appliquée à grande échelle, on parle de *monorepo*.
   - **Avantages :** Facile d'assurer que chaque version de A contient une version compatible de B.
   - **Inconvénients :** Le dépôt A peut facilement devenir énorme et désagréable à gérer. Il est difficile de mettre plusieurs équipes d'accord pour travailler dans un dépôt commun.
2. Une variante de la technique ci-dessous est de fournir, au sein du dépôt A, une copie d'une version compatible de la bibliothèque B. On parle alors de *vendoring*.
   - **Avantages :** Plus léger que l'approche précédente, car on n'a pas besoin de stocker toutes les versions de B au sein du dépôt A, il suffit d'en changer de temps en temps.
   - **Inconvénients :** Le dépôt tend quand même à grossir sur des grosses applications. On doit mettre à jour la version incluse de B régulièrement. Il faut que la licence de B nous autorise à redistribuer des copies.
3. Plutôt qu'une copie de B, on peut intégrer au processus de construction de l'application A un mécanisme automatisé qui vérifie qu'une version compatible de la bibliothèque B est présente sur le système, et éventuellement propose d'installer une version compatible si ce n'est pas le cas. Il existe un grand nombre de variantes de cette approche, du simple script shell qui fait un `wget` d'une version compatible de la dépendance à des gestionnaires de dépendances sophistiqués comme NPM, Spack ou Cargo.
   - **Avantages :** Dépôt léger. Seule approche qui permet d'exprimer clairement qu'on est compatible avec plusieurs versions différentes de B et de gérer les installations de B effectuées par l'utilisateur.
   - **Inconvénients :** Nécessite un savoir-faire spécifique à chaque projet, car les outils ont un domaine d'application limité.
4. Et enfin, on peut faire un compromis entre les approches 2 et 3 en intégrant au dépôt A un pointeur vers une version compatible du dépôt B, que

l'on peut faire évoluer vers des versions plus récentes à notre guise. Git fournit en standard un moyen de faire ça lorsque A et B sont tous les deux versionnés avec git : les *submodules*.

- **Avantages :** Dépôt léger. Intégré à git et aux forges. Simplifie la gestion de correctifs internes si besoin.
- **Inconvénients :** Exotique pour la plupart des utilisateurs de git et peu ergonomique dans l'absolu. On retrouve la nécessité de maintenir à jour la version de B sur laquelle on pointe. Ne fonctionne pas pour les logiciels non versionnés avec git.

On voit que de façon générale, l'utilisation d'un gestionnaire de dépendances dédié (approche 3) est préférable, mais que c'est aussi l'approche qui nécessite le plus de travail. Lorsqu'on n'a pas le temps ou pas la main d'œuvre qualifiée pour une telle approche, mais ne peut pas se permettre d'utiliser une approche de *vendoring*, l'utilisation de *submodules* peut parfois être un compromis intéressant.

Une bonne première approche des *submodules*, c'est de les voir comme un pointeur vers un commit précis d'un autre dépôt que l'on peut déplacer avec des commandes git. Pour fixer les idées, appelons dépôt parent le dépôt git qui contient un *submodule*.

On ajoute un *submodule* avec la commande `git submodule add`, qui prend en paramètre l'URL d'un dépôt git. Celle-ci a un effet très proche de celui de `git clone`, à ceci près qu'elle configure git pour traiter le clone nouvellement créé comme un *submodule* :

- Elle ajoute trois lignes au fichier spécial `.gitmodules`, situé à la racine du dépôt parent, qui indique quels dossiers sont des *submodules* et à quel dépôt distant (URL) ils se réfèrent.
- Elle gère le dossier associé au *submodule* de façon spéciale au niveau du dépôt parent, d'abord en permettant le stockage d'un dossier vide (ce que git n'autorise normalement pas) et ensuite en y associant une métadonnée indiquant sur quel commit du *submodule* on pointe.

Si par la suite quelqu'un clone le dépôt parent sans précautions particulières, il aura un dossier vide à l'emplacement du *submodule*. Par défaut, `git clone` ne prend pas en charge les *submodules*. Pour que ce soit fait, il faut ensuite appeler la commande `git submodule init` pour que git traite le dossier comme un *submodule*, puis `git submodule update` pour que les commits du *submodule* soient téléchargés.

Comme c'est rébarbatif et on a rarement besoin de dévier des paramètres par défaut de ces commandes (qui permettent notamment de n'initialiser qu'une partie des *submodules* du dépôt), il existe deux niveaux de raccourcis :

- `git submodule update --init --recursive` combine les fonctions de `git submodule init` et `git submodule update`, et répète les opérations dans les éventuels *submodules* des *submodules*.

- `git clone --recurse-submodules` fonctionne comme vous aimeriez probablement que le `git clone` de base fonctionne, en initialisant et clonant automatiquement les *submodules*, là encore de façon récursive.

Une fois que l'on a un projet avec des *submodules* correctement configurés, la chose la plus simple que l'on puisse faire est de basculer sur une autre version du *submodule*. Pour cela, on peut se rendre dans le dossier représentant le *submodule* et d'y utiliser des commandes git normales comme `git fetch` et `git merge origin/main`.

Par la suite, au niveau du dépôt parent, le changement de version du *submodule* sera visible dans `git status` et on pourra enregistrer ce changement de version au niveau du dépôt parent en faisant un `git add` du dossier contenant le *submodule* suivi d'un `git commit`.

Comme c'est un peu rébarbatif de se rendre dans le dossier de son *submodule* pour faire le travail, il existe un raccourci, la commande `git submodule update --remote`, qui par défaut gère le cas courant où on veut télécharger et faire un `git checkout` de la dernière version de la branche par défaut du *submodule*. Cette commande offre quelques autres possibilités, consultez `git help submodule update` pour plus de détails.

Par défaut, les commandes git ne disent pas grand-chose sur les *submodules*, mais on peut les amener à en dire davantage :

- `git diff --submodule` permet d'intégrer les modifications de *submodule* à `git diff`. Ce réglage optionnel peut être rendu permanent avec `git config --global diff.submodule log`.
- Avec `git log -p --submodule`, on peut avoir cette même information sur des commits précédents.
- Avec `git config --global status.submodulesummary 1`, on peut demander à ce que `git status` affiche la liste des nouveaux commits dans chaque *submodule*, information qui est sinon accessible via `git submodule summary`.

Par défaut, les commandes qui modifient le répertoire de travail, comme `git checkout` et `git pull`, n'affectent pas les *submodules*. Il y a deux manières de changer ça :

1. On y pense à l'avance et on ajoute l'option `--recurse-submodules` à la commande concernée avant de l'exécuter.
2. On lance `git submodule update --init --recursive` après l'opération.

Il est possible de configurer git pour qu'il travaille automatiquement en mode `--recurse-submodules` avec toutes les commandes qui le supportent à l'exception de `git clone`. Mais ce n'est pas forcément une bonne idée, car cette option peut avoir des conséquences parfois surprenantes.

Ainsi, `git pull --recurse-submodule` va basculer l'ensemble des *submodules* sur la dernière version du dépôt cible, ce qui n'est pas forcément ce qu'on veut quand les *submodules* sont utilisés pour indiquer avec quelle version du dépôt cible le dépôt parent est compatible.

Par ailleurs, on peut parfois se retrouver dans une situation où un *submodule* a changé d'URL. Ce n'est pas quelque chose que `git submodule` gère très bien, même quand on utilise manuellement `update`, il faut avoir dans ce cas recours à la commande `git submodule sync --recursive`.

Comme vous pouvez le constater, `git submodule` n'est vraiment pas un outil très ergonomique. Il est trop facile de faire des erreurs ou ne pas comprendre ce qui se passe quand on l'utilise, et la plupart du temps, quand on aurait envie d'utiliser des *submodules*, il vaut mieux commencer par évaluer les alternatives possibles.

Néanmoins, il existe une possibilité offerte par les *submodules* qui est relativement difficile à répliquer avec d'autres techniques, c'est celle de pouvoir modifier localement une dépendance en ajoutant des commits sur le *submodule* associés, avec la possibilité de mettre par la suite ces commits à jour avec des fusions et des *rebases* des nouveautés du dépôt parent.

Cette fonctionnalité étant encore plus obscure que les *submodules* eux-mêmes et un peu longue à expliquer ici, je vous invite pour plus d'informations à consulter le chapitre « Submodules » du livre Pro Git, sur lequel cette introduction est basée : https://git-scm.com/book/en/v2/Git-Tools-Submodules.